

ATROPOS CODEX-CLI BUILD BLUEPRINT (OVER TIME)

Operating model: Codex CLI is the implementation worker. Source docs are authority. Git commits are memory checkpoints. Compile/smoke gates are truth. You are Director until ATROPOS can assume Director duties.

Universal batch workflow: source-doc target -> current-code contract audit -> phase file list -> rm+nano full-file replacements or guarded tiny patches -> one slice compile -> repair real compiler errors -> wider compile -> smoke -> jar install -> commit -> export next context.

Compile rule: no compile after every small edit. Compile only after coherent slice boundaries. No success echo unless the whole gate passes.

Phase 0: Baseline Lock Goal: freeze current truth after Pass 4. Batches: status clean, current jar smoke, source context export, command registry inventory, provider symbol inventory. Done: clean repo, known commit, Pass 5 context ready.

Phase 1: Provider Activation Doctor Goal: make configured providers real and diagnosable. Batches: SecretSource/key setup script; /keys doctor backend; provider verification model; provider activation renderer; /providers verify/live-test router wiring; doctor smokes. Done: missing/configured/verified/invalid/rate-limited/locked/off states render truthfully.

Phase 2: Provider Transport Completion Goal: actual responses from real providers. Batches: OpenAI-compatible base adapter; Groq; OpenRouter; DeepInfra; SiliconFlow; Gemini; GitHub Models; Cloudflare AI; Jina; SerpAPI; Ollama; paid-locked OpenAI/Anthropic/xAI only behind explicit unlock. Done: typing hi routes to an eligible verified provider and returns a response.

Phase 3: Quota Ledger + Route Truth Goal: free-first routing that never collapses on quota. Batches: SQLite quota ledger; cooldown tracking; reset windows; skipped-provider reasons; fallback chain renderer; /status route; /status quota. Done: ATROPOS explains selected provider, skipped providers, fallback, cooldown, paid lock.

Phase 4: Secret and Security Hardening Goal: no key leaks anywhere. Batches: central redactor; secret scan before commit; command history redaction; provider auth payload redaction; vault/env precedence; /keys setup. Done: secrets never appear in UI, logs, prompts, diffs, history, or status.

Phase 5: Provider Fixture Matrix Goal: all providers testable without real keys. Batches: success fixtures; auth failure fixtures; rate-limit fixtures; malformed/empty fixtures; timeout fixtures; dry-run fixtures. Done: provider tests pass without network and live tests require opt-in.

Phase 6: DLOI Source Router Goal: source docs become addressable machine truth. Batches: DLOI coordinate model; source map loader; section resolver; document address lookup; “address never blindly ingest” enforcement. Done: ATROPOS can route a task to exact source sections.

Phase 7: AST Symbol Graph Goal: codebase awareness without guessing. Batches: Kotlin source scanner; package/path invariant; symbol node model; dependency refs; byte offsets; import reconciler. Done: ATROPOS can identify exact files/symbols impacted by a task.

Phase 8: Deterministic Verifier Goal: verify before LLM review. Batches: changed-file verifier; import verifier; public contract verifier; descriptor verifier; terminal snapshot verifier; shell safety verifier. Done: structural errors are caught locally before provider calls.

Phase 9: Persistent Memory Goal: ATROPOS remembers work across sessions. Batches: SQLite thread/session store; batch records; route history; failure signatures; successful repair memory; DLOI-linked notes; compaction. Done: restart does not erase project state or learned failures.

Phase 10: Execution Policy Engine Goal: local autonomy with boundaries. Batches: policy model; command allow/deny classes; mutation guards; destructive dry-run; audit log; approval modes. Done: every tool action is governed and logged.

Phase 11: Self-Build Loop Goal: ATROPOS can perform our current workflow. Batches: plan record; file territory; patch proposal; compile runner; stderr slicer; focused repair loop; smoke runner; commit/export planner. Done: ATROPOS can land a bounded batch with $E(\Delta)=0$.

Phase 12: Director Advisory Mode Goal: begin hierarchy without breaking current workflow. Batches: task dispatch records; territory metadata; Director status; diff monitor advisory warnings; /status hierarchy. Done: out-of-scope edits are detected and reported, not yet blocked.

Phase 13: Territory Enforcement Goal: make scope deterministic. Batches: allowed path enforcement; forbidden path checks; territory expansion requests; rollback records; paused task state. Done: autonomous work cannot silently mutate unrelated code.

Phase 14: HR Router Goal: controlled information flow. Batches: cross-territory request model; redaction policy; scope validation; approval/denial logs; narrow response channel. Done: agents do not freely leak context across boundaries.

Phase 15: Auditor and Custodian Goal: independent verification and hygiene. Batches: Auditor read-only checks; Custodian cleanup; temp artifact rules; dead state cleanup; post-batch hygiene report. Done: every accepted batch is verified and cleaned.

Phase 16: Manager/Specialist/Worker Hierarchy Goal: scalable organization. Batches: role registry; capability tags; Manager domain queues; Specialist high-risk tasks; Worker bounded execution; escalation paths. Done: ATROPOS has a deterministic software organization model.

Phase 17: Multimodal Runtime Goal: native multimodal development engine. Batches: image input; screenshot review; terminal snapshot ingestion; optional browser actuation; vision provider route; asset pipeline. Done: ATROPOS can inspect visual outputs, not just text.

Phase 18: Multiplatform Expansion Goal: CLI becomes broader native product. Batches: core extraction; renderer abstraction; Compose Desktop; Android shell; Docker/GraalVM; Ktor backend; web surface later. Done: terminal remains classic mode, core becomes portable.

Phase 19: App Factory Completion Goal: user asks for software, ATROPOS builds verified artifacts. Batches: app spec parser; DAG planner; file generator; compile/test loop; screenshot smoke; artifact packager; install/run proof. Done: prompt -> plan -> code -> verify -> artifact -> commit.

Phase 20: Full Autonomous ATROPOS Goal: self-directed, policy-bound, persistent software-development engine. Batches: autonomous backlog; scheduled doctor; queued provider retries; self-improvement scoring; route optimization; territory-governed self-build. Done: ATROPOS can improve itself through the same gates we used to build it.

Codex CLI usage pattern for every batch: Give Codex one phase batch, exact files, source-doc coordinates, current signatures, forbidden edits, compile command, smoke command, and done criteria. Let it patch and repair until green. Commit. Export next context. Repeat.

That is the over-time blueprint. Codex can write the code, but the system succeeds only if each phase is bounded, verified, committed, and carried forward as persistent project truth.

ATROPOS 100% COMPLETION BLUEPRINT — TOTAL PRODUCT

TARGET AND BUILD PATH

1. PRODUCT IDENTITY: ATROPOS is a local-first, native, persistent, multimodal, multi-provider, multi-agent, deterministic software-development engine and system-organization runtime. It is not “a chat app.” It is not “a wrapper around providers.” It is a sovereign CLI/TUI/app-factory control plane where providers are replaceable reasoning backends and ATROPOS owns routing, memory, verification, tools, files, policy, secrets, execution, and acceptance.

2. CORE THESIS: Providers do not own ATROPOS. ATROPOS owns provider access. The local runtime decides what context is sent, what tools are exposed, what files are mutable, what route is legal, what output is accepted, and what memory persists.

3. SUPERIORITY TARGET: ATROPOS must beat Codex CLI, Claude Code teams, Antigravity, and open-source swarms by combining their useful strengths while avoiding their contradictions: no uncontrolled chatty coordination, no blind long-context ingestion, no provider dependency, no reactive-only review, no hidden tool authority, no paid-auto behavior, no unbounded scope drift.

4. FORMAL INVARIANTS: $HIG=0$; $E(\Delta)=0$; $LOCAL_FIRST=true$; $PAID_AUTO=false$; $RAW_SECRET_OUTPUT=false$; $ROUTE_EXPLAINABLE=true$; $PROVIDER_REPLACEABLE=true$; $TERRITORY_BOUND=true$; $VERIFY_BEFORE_COMMIT=true$; $ADDRESS_NEVER_BLINDLY_INGEST=true$.

5. DLOI PRINCIPLE: Every source document, code symbol, subsystem, task, and memory record resolves through addressable coordinates. No cosine-RAG guessing as source truth. DLOI domain routes intent to exact source bands, exact files, exact AST symbols, exact dependency slices.

6. LOCAL ROOT: Termux, Kotlin/JVM, local filesystem, git, kotlinc, SQLite/JSONL, AST parsing, shell bridge, ANSI renderer, local verification, and local memory are mandatory root. Supabase, Google, GitHub, Cloudflare, OpenAI, Anthropic, Groq, Gemini, Ollama, and every other service are adapters.

7. PROVIDER MODEL: Providers are classified by capability, cost mode, quota state, key state, verification state, transport state, model state, fallback chain, and route priority. Configured does not mean verified. Verified does not mean always ready. Ready means configured+verified+allowed+not cooling+not locked.

8. PROVIDER STATES: missing, configured, dry-run-capable, fixture-backed, live-transport-implemented, verified, auth-failed, invalid-key, rate-limited, quota-exhausted, billing-required, offline, degraded, locked, optional, disabled, ready.

9. ROUTE LAW: LOCAL_TOOLCHAIN -> FREE_READY_PROVIDER -> FREE_FALLBACK_PROVIDER -> COOLDOWN_QUEUE -> OFFLINE_DEGRADED_MODE -> PAID_EMERGENCY_ONLY_BY_EXPLICIT_UNLOCK.

10. CLI SURFACE COMPLETE: /help, /dashboard, /status, /status quota, /status route, /status failures, /status adapters, /status endpoints, /providers, /providers descriptors, /providers validate, /providers verify ID, /providers verify all, /providers live-test ID, /keys doctor, /keys setup, /keys status, /route, /use, /use auto, /verify narrow, /verify wide, /paid status, /paid unlock, /paid lock, /factory status, /factory plan, /factory run, /memory, /shell, !cmd, /pwd, /cd, /ls, /git status, /exit.

11. UI COMPLETE: reactive terminal dashboard, responsive Termux layout, live width reflow, cards by semantic priority, lower useful panels, provider activation screen, route inspector, command palette, slash suggestions, persistent screens, tabs, tab bar, prompt preservation, bounded shell output, clean headless output, NO_COLOR support.

12. SHELL COMPLETE: shell is explicit, bounded, redacted, cwd-scoped, timeout-controlled, nonzero-aware, and never silently mixed with provider reasoning. Shell output carries cwd, command, exit code, elapsed time, stdout/stderr tail, and redaction.

13. SECRET COMPLETE: local secret setup script prompts in terminal, hidden input, chmod 600, ignored env/vault file, no chat paste, no git commit, no transcript leak, no command history leak, no raw provider auth payload leak, redaction before UI/log/persistence/prompt/queue/status.

14. PROVIDER DOCTOR COMPLETE: /keys doctor explains key source and impact; /providers verify ID checks descriptor, adapter, key source, dry-run, live gate, last normalized failure; /providers verify all summarizes operational level; /providers live-test ID is explicit opt-in only.

15. ADAPTER COMPLETE: every provider has descriptor, capabilities, env names, cost mode, quota tier, free/paid model split, fallback list, dry-run, fixture success, fixture auth failure, fixture rate limit, malformed response, empty response, timeout, normalized failure, redaction test, route eligibility, status renderer.

16. ERROR COMPLETE: raw HTTP/provider failures normalize into auth_failed, rate_limited, quota_exhausted, billing_required, model_missing, timeout, unavailable, malformed_response, empty_response, cancelled, internal. UI prints actionable local remediation, not raw JSON.

17. QUOTA COMPLETE: SQLite quota ledger records provider_id, cost_mode, configured, verified, state, active_model, quota windows, used/remaining requests/tokens, reset, cooldown, last success/failure, normalized error class, latency, success score, paid lock, fallback chain.

18. MEMORY COMPLETE: SQLite/JSONL persistent memory stores thread/session state, batch records, route outcomes, compile failures, fixed errors, source-doc decisions, DLOI links, tool results, reward/penalty vectors, and forgotten/compacted summaries.

19. VERIFICATION COMPLETE: deterministic verifier parses changed files, checks package/path invariant, imports, public contracts, route policy, provider descriptors, redaction, terminal snapshots, shell safety, command registry integrity, build compatibility, and only escalates to LLM when deterministic checks cannot decide.

20. SELF-IMPROVEMENT COMPLETE: every build/test/provider/tool result becomes a structured reward signal. Success and failure are attributed to route, provider, prompt, batch, territory, file slice, and invariant. ATROPOS improves prompts and route choices from local evidence, not blind memory.

21. MULTI-AGENT COMPLETE: ATROPOS does not copy chatty swarms. It uses hierarchy plus territory. Director owns global visibility; Managers own domain queues; Specialists execute high-risk narrow tasks; Workers execute bounded tasks; Auditor verifies independently; Custodian cleans deterministic state; HR Router controls cross-boundary information flow.

22. TERRITORY COMPLETE: every autonomous task has allowed paths, forbidden paths, dependency inputs, verification gates, rollback plan, and diff monitor. Out-of-territory edits are logged in advisory mode first, then blocked in enforcement mode.

23. WHY BETTER THAN CODEX CLI: Codex has strong sandbox/thread primitives but weak native multi-agent territory enforcement. ATROPOS adds DLOI, territory metadata, Director diff monitoring, provider quota routing, local-first memory, and product-native hierarchy.

24. WHY BETTER THAN CLAUDE TEAMS: Claude Teams scale by LLM-mediated messages and external harnesses. ATROPOS moves coordination into deterministic state: task queues, territories, policy, diff checks, and HR routing. Less token waste, less drift, clearer authority.

25. WHY BETTER THAN ANTIGRAVITY: Antigravity has strong agent/browser UX but remains platform/product controlled. ATROPOS is local-first, AGPL-protected, provider-agnostic, terminal-native first, and eventually multiplatform.

26. WHY BETTER THAN OPEN-SOURCE SWARMS: Most swarms have worktrees and mailboxes but weak DLOI, weak deterministic verification, weak free-first provider control, weak secret discipline, weak source-doc authority. ATROPOS makes those core primitives.

27. BUILD WORKFLOW: Plan in phases. Build in tiers. Land in batches. Repair in steps. Compile narrowly. Widen only after green. Commit only at stable checkpoints. Export next context immediately.

28. PATCH WORKFLOW: Read current contracts first. Use rm+nano for full-file replacements. Use guarded python one-liners only for tiny anchor-proven edits. No manual wandering edits. No Perl. No blind insertions. No compile after every small line. Compile after coherent backend/UI/provider slices.

29. ACCEPTANCE WORKFLOW: source truth check -> patch batch -> narrow slice compile -> focused smoke -> wider compile -> full jar compile -> CLI smoke -> diff check -> secret scan -> safe jar swap -> installed smoke -> commit -> clean ignored runtime -> final clean status -> export next context.

30. MACRO 0 COMPLETE BASELINE: Keep existing committed shell bridge, slash UX, screens/tabs, responsive dashboard, provider descriptors, route/quota primitives, adapter kernel, and current jar checkpoint as the foundation.

31. MACRO 1 PROVIDER ACTIVATION: finish /keys doctor, setup script, provider activation renderer, /providers verify, /providers verify all, /providers live-test, route inspector, missing/invalid/rate-limit diagnostics, provider doctor smokes.

32. MACRO 2 PROVIDER TRANSPORT: implement schema-compatible OpenAI-compatible transport group first: groq, openrouter, deepinfra, siliconflow. Then individual providers: gemini, github_models, cloudflare_ai, jina, serpapi, huggingface, ollama, paid-locked providers last.

33. MACRO 3 QUOTA AND ROUTING: capability-specific route policy, cooldown queue, free-only default, paid emergency gate, route explanation, skipped-provider table, last-route persistence, fallback chain proving.
34. MACRO 4 SECRET/Vault HARDENING: SecretSource interface everywhere, env/local vault/cloud vault precedence, redaction filter centralization, setup doctor, git secret scan, no raw keys in UI/log/model prompts.
35. MACRO 5 DLOI/AST INDEX: implement DLOI router, AST symbol graph, package/path invariant checker, import reconciler, symbol dependency graph, changed-slice extractor, context slicing by exact address.
36. MACRO 6 VERIFICATION MATRIX: terminal snapshot tests at 40/80/120/160, provider fixture tests, route tests, shell tests, input tests, descriptor validation, redaction tests, compile lanes, invariant names, failure output with observed values.
37. MACRO 7 MEMORY ENGINE: persistent threads, session tabs, project state, batch records PLAN/FILES/VERIFY/ROLLBACK, failure signatures, successful patch memory, DLOI-linked decisions, compaction, resume state.
38. MACRO 8 EXECUTION POLICY: project-local rules DSL, tool permissions, command approval classes, mutation guards, destructive dry-run/yes gates, audit log for every file write, shell call, provider call, route, and verification result.
39. MACRO 9 SELF-BUILD LOOP: ATROPOS can plan a batch, assign territory, generate patch, compile slice, parse stderr, repair only failed invariant, widen compile, smoke, stage, commit, export context, and log reward.
40. MACRO 10 HIERARCHY ADVISORY: add Director task records, territory metadata, diff monitor advisory mode, /status hierarchy, /status territory, drift alerts, no enforcement yet.
41. MACRO 11 HIERARCHY ENFORCEMENT: enforce territory boundaries, pause out-of-scope workers, require territory expansion request, route cross-boundary info through HR Router, log approvals/denials.

42. MACRO 12 ROLE SYSTEM: Managers for domains, Specialists for high-risk capabilities, Workers for bounded execution, Auditor read-only verification, Custodian cleanup/state hygiene, VP layer only after Manager layer proves useful.

43. MACRO 13 MULTIMODAL: image/screenshot intake, terminal screenshot review, optional browser actuation, UI visual regression, local asset generation first, remote vision/image providers only via provider policy.

44. MACRO 14 MULTIPLATFORM: extract pure core, keep CLI classic mode, Compose Desktop dashboard, Android app shell, Docker/GraalVM packaging, Ktor backend if needed, web UI later, iOS only after core is stable.

45. MACRO 15 LAKEHOUSE/CLOUD: local source docs first, Drive mirror optional, CAS exports, GCP/Firebase/Supabase/Pinecone optional adapters, no cloud as foundation, OAuth secrets isolated, uploads audited.

46. MACRO 16 APP FACTORY: natural-language app request -> DLOI/source lookup -> DAG plan -> provider route -> code batch -> verification -> runnable artifact -> smoke -> screenshot/output proof -> commit/export.

47. MACRO 17 AUTONOMY LEVELS: suggest/read-only, auto-edit within territory, full local auto with policy, live provider opt-in, paid emergency opt-in, destructive operations gated, network governed by provider policy.

48. MACRO 18 FINAL ACCEPTANCE: ATROPOS starts without cloud keys, answers with configured provider when keys exist, edits only through local policy, routes around quota, verifies before commit, survives restart, preserves memory, enforces territories, redacts secrets, and can build verified software batches.

49. TIER ORDER WITHIN EVERY MACRO: Tier 0 contracts/models; Tier 1 deterministic utilities; Tier 2 adapters/engines; Tier 3 orchestration; Tier 4 CLI/UI exposure; Tier 5 smokes/tests; Tier 6 jar/install/commit/export.

50. BATCH SIZE RULE: 3-8 files, 500-2000 LOC, one dependency slice, one architectural promise, one rollback boundary. Large is allowed. Muddy is not.

51. PROVIDER FAMILY RULE: group only identical schemas.

Groq/OpenRouter/DeepInfra/SiliconFlow may group after base adapter exists. Gemini, Anthropic, Cohere, HuggingFace, Jina, SerpAPI, Cloudflare, Supabase, Google APIs are separate unless proven compatible.

52. COMMANDROUTER RULE: never patch CommandRouter first.

Contracts/adapters/backend first, renderer/status second, command registry third, router last.

53. UI RULE: never patch provider logic and dashboard layout in the same batch unless observability requires it. Terminal UI is presentation, not adapter logic.

54. COMPILE RULE: no success echo after failed compile. SUCCESS appears only after compile, smoke, grep truth checks, diff check, jar swap, and installed smoke all pass.

55. SECRET RULE: before every commit, scan staged diff for keys, bearer tokens, OAuth tokens, client secrets, private keys, signed URLs, and credential paths.

56. MATHEMATICAL COMPLETION SCORE: $\text{product_progress} = \text{accepted_macros} / \text{planned_macros}$; $\text{batch_valid} = \text{compile_green} \text{ AND } \text{smoke_green} \text{ AND } \text{secrets_zero} \text{ AND } \text{diff_scope_valid} \text{ AND } \text{E}(\text{DELTA})=0$; $\text{provider_ready} = \text{configured} \text{ AND } \text{verified} \text{ AND } \text{allowed} \text{ AND } \text{not_cooling} \text{ AND } \text{not_locked}$; $\text{autonomy_safe} = \text{territory_bound} \text{ AND } \text{policy_logged} \text{ AND } \text{rollback_available}$.

57. FINAL PRODUCT IMAGE: ATROPOS opens as a terminal-native command deck showing dashboard, providers, route, keys, memory, factory, tasks, logs, verification, tabs, and shell. User types “build X” or “fix Y.” ATROPOS routes locally first, chooses providers only as needed, sends exact context slices, generates bounded patches, verifies deterministically, asks only when policy requires, commits clean checkpoints, remembers failures, and improves.

58. FINAL CLAIM: ATROPOS is complete only when it can act as a deterministic local software-development organization: provider-agnostic reasoning, local execution, persistent memory, source-addressed context, verified file mutation, policy-governed autonomy, role hierarchy, territory enforcement, multimodal inspection, quota-aware routing, and rollback-safe self-build loops.

59. NEXT PRACTICAL MOVE: finish Macro 1 Provider Activation with the fast batch workflow, because keys/provider truth are the ignition layer for every higher capability.

60. NEXT BLUEPRINT AFTER MACRO 1: provider transport completion, then DLOI/AST verification, then memory/policy, then self-build loop, then hierarchy enforcement.